Université Hassan II de

Casablanca Faculté des Sciences
Ben M'Sick

Département Informatique

Master : Intelligence Artificielle
Niveau : M1
Année universitaire : 2025–2026

Rapport de Projet

Mise en Place d'une Infrastructure DevOps avec CI/CD, Kubernetes et GitOps

---

PROJET : INFORGANIZER

Organisateur d'informations pour candidatures universitaires

Réalisé par :
Mousdik Ismail
Edaoudi Mohamed
Jounaidi Oussama
Erfoudi Anass

Encadrant :
Pr. F. Benabbou

Casablanca, 2026

Resume

Ce rapport presente la conception, l'implementation et le deploiement d'une infrastructure DevOps complete pour une application web de gestion de candidatures universitaires. Le projet InfOrganizer illustre l'integration des pratiques modernes du genie logiciel dans un contexte cloud-native.

Objectif principal : Automatiser l'ensemble du cycle de vie applicatif, depuis le developpement jusqu'a la production, en eliminant les interventions manuelles sources d'erreurs et en garantissant la reproductibilite des environnements.

Approche methodologique : L'architecture repose sur le paradigme GitOps, ou Git constitue la source de verite unique pour l'etat desire du systeme. Cette approche, combinee a l'Infrastructure as Code (IaC), assure une tracabilite complete et une capacite de rollback instantanee.

Implementation technique :

- **Infrastructure** : AWS (VPC, EC2) provisionnee via Terraform
- **Configuration** : Cluster Kubernetes k3s deploye par Ansible
- **Conteneurisation** : Docker avec build multi-stage et principe du moindre privilege
- **CI/CD** : GitHub Actions avec authentification OIDC (zero credentials statiques)
- **Deploiement** : ArgoCD pour la reconciliation GitOps automatique

Resultats obtenus : Le systeme atteint un temps de deploiement de 1 minute 22 secondes entre le commit et la mise en production, avec zero intervention manuelle. La haute disponibilite est assuree par 2 replicas avec anti-affinite et strategie RollingUpdate.

Mots-cles : DevOps, GitOps, Kubernetes, CI/CD, Terraform, Ansible, Docker, AWS, ArgoCD, OIDC

Table des matières

Resume **1**

1 Introduction **6**
1.1 Contexte et motivation 6
1.2 Problematique 6
1.3 Objectifs 7
1.4 Perimetre du projet 7

2 Architecture du Systeme **8**
2.1 Vision architecturale 8
2.2 Vue logique de l'architecture 9
2.3 Flux de deploiement detaille 9
2.4 Architecture reseau AWS 10

3 Technologies et Outils **11**
3.1 Choix technologiques et justifications 11
3.1.1 Docker : Conteneurisation 11
3.1.2 Kubernetes (k3s) : Orchestration 12
3.1.3 Terraform : Infrastructure as Code 12
3.1.4 Ansible : Configuration Management 13
3.1.5 GitHub Actions : CI/CD 13
3.1.6 ArgoCD : GitOps 14

4 Details d'Implementation **15**
4.1 Application InfOrganizer 15
4.1.1 Description fonctionnelle 15
4.1.2 Architecture technique 15
4.1.3 Schema de base de donnees 15
4.2 Deployment Kubernetes 16
4.2.1 Strategie de deploiement 16
4.2.2 Init Container : Attente de PostgreSQL 16
4.2.3 Anti-affinite pour haute disponibilite 17
4.2.4 Probes de sante 17
4.2.5 Gestion des secrets 18
4.3 Pipeline CI/CD detaille 19
4.3.1 Job 1 : Tests 19
4.3.2 Job 2 : Build et Push ECR 19
4.3.3 Job 3 : Mise a jour GitOps 19

5 Processus de Deploiement **21**

5.1 Procedure de deploiement initial . . . 21
5.1.1 Phase 1 : Provisionnement AWS (Terraform) . . . 21
5.1.2 Phase 2 : Configuration du cluster (Ansible) . . . 21
5.1.3 Phase 3 : Verification . . . 22
5.2 Deploiement continu automatise . . . 22

6 Preuves et Validation **23**
6.1 Infrastructure AWS . . . 23
6.1.1 Instances EC2 . . . 23
6.1.2 Security Groups . . . 23
6.1.3 Configuration IAM OIDC . . . 24
6.2 Provisionnement et Configuration . . . 25
6.2.1 Terraform Apply . . . 25
6.2.2 Ansible Playbook . . . 25
6.3 Cluster Kubernetes . . . 26
6.3.1 Etat des noeuds . . . 26
6.3.2 Pods applicatifs . . . 26
6.3.3 Services exposes . . . 26
6.3.4 Test de sante de l'API . . . 27
6.4 Pipeline CI/CD . . . 27
6.4.1 Execution reussie . . . 27
6.4.2 Image publiee sur ECR . . . 27
6.5 GitOps ArgoCD . . . 28
6.5.1 Synchronisation . . . 28
6.6 Tableau recapitulatif des validations . . . 29

7 Discussion **30**
7.1 Points forts de l'implementation . . . 30
7.1.1 Automatisation complete . . . 30
7.1.2 Securite en profondeur . . . 30
7.1.3 Haute disponibilite . . . 30
7.2 Limites identifiees . . . 30
7.2.1 Single Point of Failure : Control Plane . . . 30
7.2.2 Absence de monitoring . . . 31
7.2.3 Gestion des secrets . . . 31
7.2.4 Backup de la base de donnees . . . 31
7.3 Ameliorations proposees . . . 31

8 Conclusion **32**
8.1 Synthese des realisations . . . 32
8.2 Competences acquises . . . 32
8.3 Perspectives . . . 33
8.4 Mot de fin . . . 33

Bibliographie **34**

A Annexes **35**
A.1 Structure complete du projet . . . 35
A.2 Commandes de reference . . . 35

A.2.1 Terraform 35
A.2.2 Ansible 36
A.2.3 Kubernetes 36
A.2.4 ArgoCD 36
A.2.5 Docker 36
A.3 Variables d'environnement requises 37

Table des figures

2.1 Architecture GitOps du systeme 9

6.1 Instances EC2 provisionnees sur AWS 23
6.2 Configuration des Security Groups AWS 24
6.3 Configuration du role IAM OIDC pour GitHub Actions 24
6.4 Execution reussie de Terraform apply 25
6.5 Recap de l'execution du playbook Ansible 25
6.6 Noeuds Kubernetes en etat Ready 26
6.7 Pods applicatifs en execution 26
6.8 Services Kubernetes exposes 26
6.9 Test de sante de l'API - reponse OK 27
6.10 Pipeline CI/CD execute avec succes 27
6.11 Image Docker publiee sur AWS ECR 28
6.12 ArgoCD - Application synchronisee et saine 28

Liste des tableaux

1.1 Objectifs du projet et criteres de succes 7

2.1 Configuration reseau et justifications 10

3.1 Comparaison des distributions Kubernetes 12

4.1 Stack technique de l'application 15

6.1 Synthese des validations 29

7.1 Ameliorations proposees par priorite 31

A.1 Variables d'environnement et secrets 37

Chapitre 1

Introduction

1.1 Contexte et motivation

L'industrie du logiciel a connu une transformation profonde avec l'emergence du mouvement DevOps, ne de la necessite de reduire les frictions entre les equipes de developpement et d'operations. Cette evolution repond a plusieurs defis majeurs :

— **Acceleration des cycles de livraison** : Les entreprises doivent deployer des mises a jour plusieurs fois par jour, contre quelques fois par an auparavant.

— **Complexite croissante des infrastructures** : L'adoption du cloud et des microservices multiplie les composants a gerer.

— **Exigences de fiabilite** : Les utilisateurs attendent une disponibilite quasi-permanente des services.

Dans ce contexte, ce projet vise a mettre en pratique les concepts DevOps sur un cas d'etude concret : le deploiement d'une application web full-stack dans un environnement cloud.

1.2 Problematique

Le deploiement traditionnel d'applications souffre de plusieurs limitations critiques :

1. **Erreurs humaines** : Les configurations manuelles introduisent des incoherences entre environnements.
2. **Dette technique operationnelle** : L'absence de documentation executable rend la reproduction des environnements aleatoire.
3. **Temps de deploiement** : Les processus manuels peuvent prendre des heures, voire des jours.
4. **Absence de tracabilite** : Il est difficile de savoir qui a modifie quoi et quand.
5. **Rollback complexe** : Revenir a une version anterieure necessite souvent une intervention manuelle delicate.

Question de recherche : Comment concevoir une infrastructure permettant un deploiement automatise, reproductible et auditable, tout en maintenant la securite et la haute disponibilite ?

1.3 Objectifs

Ce projet poursuit les objectifs suivants, classes par priorite :

Priorite	Objectif	Critere de succes
1	Automatisation complete	Zero intervention manuelle
2	Reproductibilite	Infrastructure recreable en $<$ 30 min
3	Securite	Pas de credentials statiques
4	Haute disponibilite	Zero downtime lors des deploiements
5	Tracabilite	Audit trail complet via Git

Table 1.1 – Objectifs du projet et criteres de succes

1.4 Perimetre du projet

Inclus dans le perimetre :

- Provisionnement de l'infrastructure cloud (AWS)
- Configuration automatisee des serveurs
- Deploiement d'un cluster Kubernetes
- Pipeline CI/CD complet
- Deploiement GitOps de l'application

Hors perimetre (perspectives d'amelioration) :

- Monitoring avance (Prometheus/Grafana)
- Gestion centralisee des secrets (Vault)
- Multi-environnements (staging/production)

Chapitre 2

Architecture du Systeme

2.1 Vision architecturale

L'architecture adoptee suit le paradigme **GitOps**, defini par Weaveworks en 2017, qui repose sur quatre principes fondamentaux :

1. **Declaratif** : L'etat desire du systeme est decrit, pas les etapes pour y parvenir.
2. **Versionne et immuable** : Git stocke l'historique complet des configurations.
3. **Tire automatiquement** : Un agent surveille Git et applique les changements.
4. **Reconciliation continue** : L'agent corrige toute derive entre l'etat desire et l'etat reel.

Cette approche a ete choisie plutot qu'un modele push traditionnel (ou le CI pousse directement vers le cluster) pour plusieurs raisons :

— **Securite** : Le CI n'a pas besoin d'acces direct au cluster Kubernetes.

— **Auditabilite** : Chaque changement est trace dans Git.

— **Self-healing** : ArgoCD corrige automatiquement les modifications manuelles non autorisees.

2.2 Vue logique de l'architecture

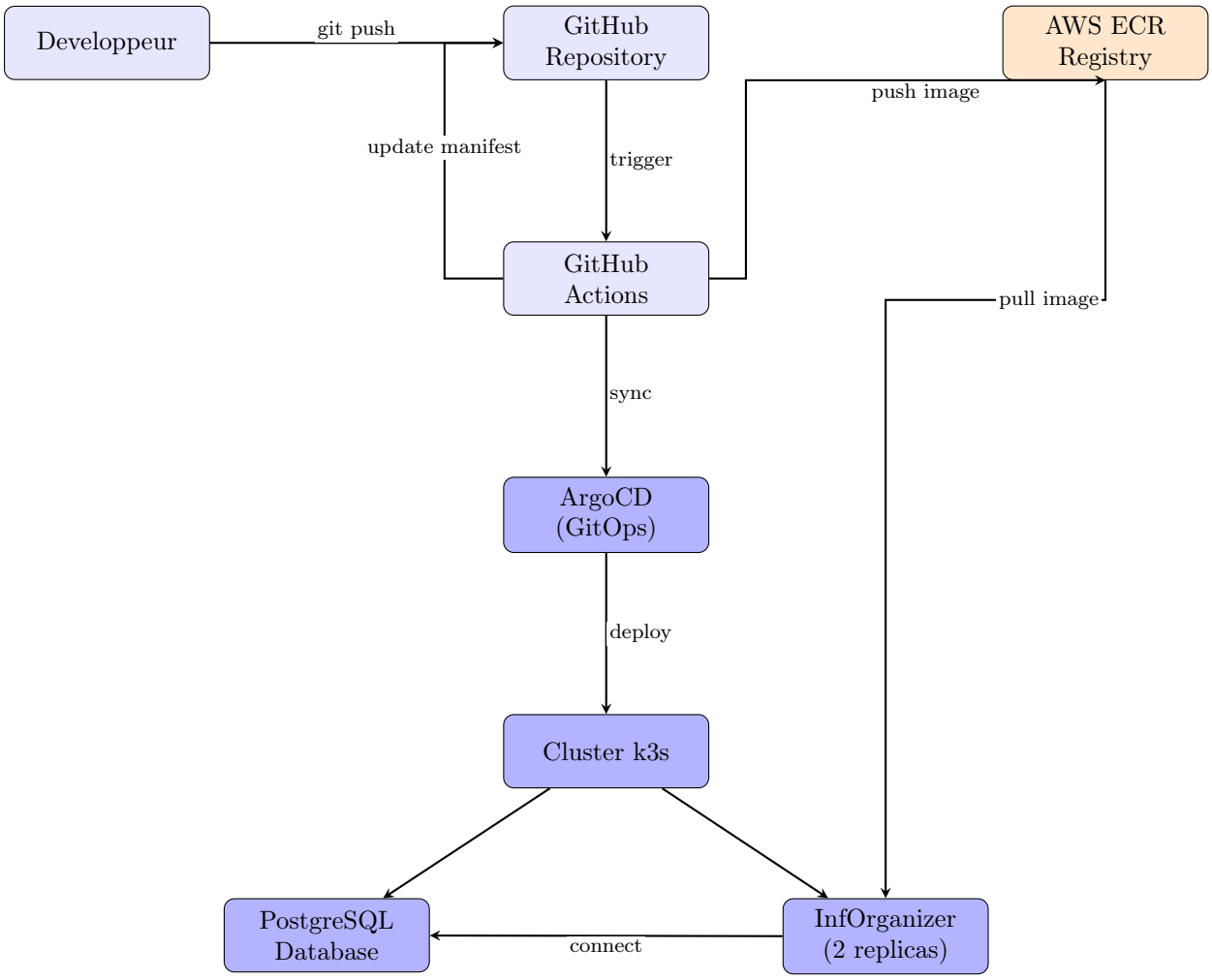

FIGURE 2.1 – Architecture GitOps du systeme

2.3 Flux de deploiement detaille

Le deploiement suit un flux en 7 etapes, chacune ayant un role precis :

1. **Commit** : Le developpeur pousse son code sur la branche `main`.
2. **Declenchement CI** : GitHub Actions demarre le workflow `ci.yml`.
3. **Tests** : Job `test` - execution des tests unitaires avec base PostgreSQL ephemere.
4. **Build** : Job `build-push-ecr` - construction de l'image Docker multi-stage.
5. **Publication** : Push de l'image vers AWS ECR avec tag SHA (ex : `sha-f650335950b4`).
6. **Mise a jour GitOps** : Job `gitops-update` - modification du tag dans `k8s/deployment.yaml`.
7. **Synchronisation** : ArgoCD detecte le changement et applique le nouveau manifest au cluster.

Pourquoi ce flux ?

— La separation en 3 jobs permet l'echec rapide : si les tests echouent, on ne gaspille pas de ressources a construire l'image.

— Le tag SHA garantit l'immutabilite : chaque commit produit une image unique et tracable.

— Le commit `[skip ci]` evite les boucles infinies de declenchement.

2.4 Architecture reseau AWS

L'infrastructure AWS est concue selon le principe de defense en profondeur :

Composant	**CIDR/Port**	**Justification**
VPC	10.0.0.0/16	Isolation reseau complete
Subnet public	10.0.1.0/24	254 adresses suffisantes
SSH (22)	allowed_cidr uniquement	Limite a l'IP admin
k3s API (6443)	allowed_cidr uniquement	Securisation du control plane
NodePort (30080)	0.0.0.0/0	Acces public a l'application
Intra-cluster	self	Communication pods libre

Table 2.1 – Configuration reseau et justifications

Chapitre 3

Technologies et Outils

3.1 Choix technologiques et justifications

Chaque technologie a ete selectionnee selon des criteres precis. Cette section explique non seulement *quoi* mais surtout *pourquoi*.

3.1.1 Docker : Conteneurisation

Pourquoi Docker ?

- **Portabilite** : "Build once, run anywhere" - l'image fonctionne identiquement en dev et en prod.
- **Isolation** : Chaque conteneur a son propre filesystem et reseau.
- **Efficacite** : Partage du kernel avec l'hote, contrairement aux VMs.

Implementation : Build multi-stage
Le Dockerfile utilise un build en deux stages pour optimiser l'image finale :

```
# Stage 1: Installation des dependances uniquement
FROM node:20-alpine AS deps
WORKDIR /app
RUN npm install -g pnpm
COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
RUN pnpm install --prod --frozen-lockfile

# Stage 2: Image finale allegee
FROM node:20-alpine
WORKDIR /app
# Securite : utilisateur non-root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=deps /app/node_modules ./node_modules
COPY server/ ./server/
COPY client/ ./client/
USER appuser
HEALTHCHECK --interval=30s --timeout=5s \
    CMD wget -qO- http://localhost:3000/api/health || exit 1
CMD ["node", "server/server.js"]
```

Listing 3.1 – Dockerfile multi-stage (extrait)

Avantages de cette approche :

1. **Taille reduite** : L'image finale ne contient pas les outils de build (npm, cache).

2. **Securite** : L'utilisateur `appuser` n'a pas les privileges root.
3. **Observabilite** : Le HEALTHCHECK integre permet a Docker et Kubernetes de verifier l'etat.
4. **Reproductibilite** : `-frozen-lockfile` garantit les memes versions de dependances.

Resultat mesure : Image de 49.31 Mo (voir preuve ECR, Figure 6.11).

3.1.2 Kubernetes (k3s) : Orchestration

Pourquoi k3s plutot que kubeadm ou EKS ?

Critere	k3s	kubeadm	EKS
Memoire minimale	512 Mo	2 Go	N/A (managed)
Temps d'installation	$<$ 1 min	15-30 min	15-20 min
Cout	Gratuit	Gratuit	0.10$/h/cluster
Compatibilite K8s	100%	100%	100%
Ideal pour	Edge, dev, small clusters	Production	Large scale

TABLE 3.1 – Comparaison des distributions Kubernetes

k3s a ete choisi car :

— Le cluster est de petite taille (2 noeuds).

— Les ressources EC2 sont limitees (c7i-flex.large, 2 vCPU, 4 Go RAM).

— L'installation scriptable s'integre parfaitement avec Ansible.

3.1.3 Terraform : Infrastructure as Code

Pourquoi Terraform plutot que CloudFormation ou Pulumi ?

— **Multi-cloud** : Terraform supporte AWS, GCP, Azure avec la meme syntaxe.

— **Etat declaratif** : Le fichier `terraform.tfstate` trace l'etat reel.

— **Plan avant apply** : `terraform plan` montre les changements avant execution.

— **Ecosysteme mature** : Providers officiels maintenus par HashiCorp et les cloud providers.

Ressources provisionnees :

```
resource "aws_vpc" "inforganizer" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "inforganizer-vpc" }
}

resource "aws_instance" "master" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "c7i-flex.large"
  key_name      = var.key_name
  root_block_device {
    volume_size = 20
    volume_type = "gp3"
  }
```

```
  tags = { Name = "inforganizer-master", Role = "master" }
}
```

Listing 3.2 – Ressources Terraform (extrait de main.tf)

Point technique important : La generation de l'inventaire Ansible utilise `null_resource` avec `local-exec` plutot que `local_file`. Ce choix evite l'erreur "Provider produced inconsistent final plan" qui survient quand les IPs ne sont pas encore connues lors du plan.

3.1.4 Ansible : Configuration Management

Pourquoi Ansible plutot que Chef ou Puppet ?

- **Agentless** : Pas de daemon a installer sur les noeuds cibles.
- **Idempotence native** : Les modules garantissent le meme resultat a chaque execution.
- **YAML lisible** : Les playbooks sont comprehensibles sans formation specifique.
- **Integration SSH** : Utilise l'infrastructure SSH existante.

Structure du playbook principal :
Le playbook `install_k3s.yml` est organise en 5 plays distincts :

1. **Common setup** (tous les noeuds) : Swap, Docker, paquets systeme.
2. **Master k3s** : Installation du serveur, recuperation du token.
3. **Worker k3s** : Jointure au cluster avec le token.
4. **Stabilisation** : Attente que tous les noeuds soient Ready.
5. **Bootstrap GitOps** : Installation ArgoCD, secrets, application.

3.1.5 GitHub Actions : CI/CD

Pourquoi GitHub Actions plutot que Jenkins ou GitLab CI ?

- **Integration native** : Pas de serveur CI a maintenir.
- **OIDC natif** : Authentification AWS sans secrets statiques.
- **Marketplace** : Actions prereates pour Docker, AWS, etc.
- **Gratuit** : 2000 minutes/mois pour les repos publics.

Securite OIDC - Pourquoi c'est crucial :
L'authentification traditionnelle AWS necessite des credentials statiques (Access Key + Secret Key) stockes en secrets GitHub. Problemes :

- Risque de fuite si le secret est expose.
- Rotation manuelle necessaire.
- Credentials valides indefiniment.

Avec OIDC :

```
permissions:
  id-token: write  # Permet de demander un token OIDC

- name: Configure AWS credentials via OIDC
  uses: aws-actions/configure-aws-credentials@v4
```

```
  with:
    role-to-assume: ${{ secrets.AWS_GITHUB_ROLE_ARN }}
    aws-region: eu-west-3
```

Listing 3.3 – Authentification OIDC (extrait de ci.yml)

Le token OIDC :

- Est genere a chaque execution (pas de stockage).
- Expire apres 1 heure maximum.
- Est lie au repository et a la branche (impossible de l'utiliser ailleurs).

3.1.6 ArgoCD : GitOps

Pourquoi ArgoCD plutot que Flux CD ?

- **UI Web** : Interface graphique pour visualiser l'etat des applications.
- **Sync policies** : Configuration fine du comportement de synchronisation.
- **RBAC integre** : Gestion des permissions par projet.
- **Multi-cluster** : Peut gerer plusieurs clusters depuis une instance centrale.

Configuration de l'application :

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: inforganizer
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/itsma3il/inforganizer-devopss.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true       # Supprime les ressources orphelines
      selfHeal: true    # Corrige les derives manuelles
```

Listing 3.4 – inforganizer-app.yaml

Comportement du self-heal : Si un operateur modifie manuellement un deployment (ex : change le nombre de replicas), ArgoCD detecte la derive et restaure l'etat defini dans Git dans les 3 secondes.

Chapitre 4

Details d'Implementation

4.1 Application InfOrganizer

4.1.1 Description fonctionnelle

InfOrganizer est une application web permettant aux etudiants d'organiser leurs candidatures universitaires. Elle offre :

- Gestion des universites (CRUD) avec statut (Applied, Pending, Accepted, Rejected)
- Stockage de notes personnalisees avec champs dynamiques
- Systeme de rappels avec dates
- Authentification securisee par JWT
- Interface bilingue (francais/anglais)

4.1.2 Architecture technique

Composant	Technologie	Role
Backend	Node.js 20 + Express	API REST
Base de donnees	PostgreSQL 16	Persistance
Authentification	JWT (jsonwebtoken)	Sessions stateless
Hachage mots de passe	bcryptjs (cost 12)	Securite
Frontend	HTML/CSS/JS vanilla	Interface utilisateur

Table 4.1 – Stack technique de l'application

4.1.3 Schema de base de donnees

```sql
CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    email         TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at    TEXT NOT NULL
);

CREATE TABLE info_notes (
    id          SERIAL PRIMARY KEY,
```

```sql
    user_id     INTEGER REFERENCES users(id) ON DELETE CASCADE,
    name        TEXT NOT NULL,
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL
);

CREATE TABLE universities (
    id          SERIAL PRIMARY KEY,
    note_id     INTEGER REFERENCES info_notes(id) ON DELETE CASCADE,
    name        TEXT NOT NULL,
    type        TEXT DEFAULT 'CI',
    status      TEXT DEFAULT 'Applied'
);

CREATE TABLE reminders (
    id      SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    text    TEXT NOT NULL,
    date    TEXT NOT NULL
);
```

Listing 4.1 – Schema PostgreSQL (simplifie)

Choix de conception :

- `ON DELETE CASCADE` : Suppression en cascade pour maintenir l'integrite referentielle.
- Dates en `TEXT` (ISO 8601) : Simplicite et compatibilite JavaScript.
- Index implicites sur les cles primaires et etrangeres.

4.2 Deployment Kubernetes

4.2.1 Strategie de deploiement

Le deployment utilise une strategie `RollingUpdate` avec des parametres specifiques :

```yaml
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0  # Toujours au moins 2 pods disponibles
      maxSurge: 1        # Au plus 3 pods pendant la transition
```

Listing 4.2 – Strategie RollingUpdate

Pourquoi ces valeurs ?

- `maxUnavailable: 0` : Garantit zero downtime - le nouveau pod doit etre Ready avant de terminer l'ancien.
- `maxSurge: 1` : Limite la consommation de ressources pendant le deploiement.

4.2.2 Init Container : Attente de PostgreSQL

```yaml
initContainers:
  - name: wait-for-postgres
    image: postgres:16-alpine
    command: ["sh", "-c",
      "until pg_isready -h postgres -p 5432; do
         echo 'waiting for postgres...'; sleep 3;
       done"]
```

Listing 4.3 – Init container pour dependance

Pourquoi un init container ?

— L'application Node.js echoue si PostgreSQL n'est pas disponible au demarrage.

— L'init container bloque le demarrage du container principal jusqu'a ce que la DB soit prete.

— Utilisation de `postgres:16-alpine` car elle contient `pg_isready` (l'image `busybox` ne l'a pas).

4.2.3 Anti-affinite pour haute disponibilite

```yaml
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: [inforganizer]
          topologyKey: kubernetes.io/hostname
```

Listing 4.4 – Pod anti-affinity

Explication :

— `preferred` (pas `required`) : Le scheduler essaie de placer les pods sur des noeuds differents, mais ne bloque pas si c'est impossible.

— `weight: 100` : Priorite maximale a cette regle.

— `topologyKey: kubernetes.io/hostname` : Distribution par noeud physique.

Benefice : Si un noeud tombe en panne, au moins un replica reste disponible sur l'autre noeud.

4.2.4 Probes de sante

```yaml
livenessProbe:
  httpGet:
    path: /api/health
    port: 3000
  initialDelaySeconds: 15
  periodSeconds: 30

readinessProbe:
```

```
  httpGet:
    path: /api/health
    port: 3000
  initialDelaySeconds: 10
  periodSeconds: 10
```

Listing 4.5 – Liveness et Readiness probes

Difference entre les deux probes :

— **Liveness** : "Le pod est-il vivant ?" - Si echec, Kubernetes redemarre le pod.

— **Readiness** : "Le pod est-il pret a recevoir du trafic ?" - Si echec, le pod est retire du Service.

Pourquoi des valeurs differentes ?

— Readiness demarre plus tot (10s vs 15s) pour detecter rapidement les pods prets.

— Liveness est moins frequent (30s vs 10s) pour eviter les faux positifs lors de pics de charge.

4.2.5 Gestion des secrets

Les secrets sensibles sont injectes via Kubernetes Secrets :

```
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: postgres-secret
        key: DATABASE_URL
  - name: JWT_SECRET
    valueFrom:
      secretKeyRef:
        name: postgres-secret
        key: JWT_SECRET
```

Listing 4.6 – Injection des secrets

Workflow de creation des secrets :

1. L'operateur definit `POSTGRES_PASSWORD` et `JWT_SECRET` en variables d'environnement locales.
2. Ansible lit ces variables via `lookup('env', 'POSTGRES_PASSWORD')`.
3. Ansible genere le manifest Secret via template Jinja2.
4. Le Secret est applique au cluster avec `kubectl apply`.

Pourquoi cette approche ?

— Les secrets ne sont jamais commites dans Git.

— Chaque environnement peut avoir des secrets differents.

— Les secrets sont chiffres au repos dans etcd (si configure).

4.3 Pipeline CI/CD detaille

4.3.1 Job 1 : Tests

```yaml
test:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
        POSTGRES_DB: inforganizer_test
      ports: ["5432:5432"]
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5
```

Listing 4.7 – Job de tests avec service PostgreSQL

Point cle : Le service PostgreSQL est demarre comme conteneur sidecar. Les health options garantissent que la DB est prete avant l'execution des tests.

4.3.2 Job 2 : Build et Push ECR

Ce job contient plusieurs validations de securite :

```bash
# Verification que le secret existe et est valide
if ! echo "${{ secrets.AWS_ACCOUNT_ID }}" | grep -qE '^[0-9]{12}$'; then
  echo "::error::AWS_ACCOUNT_ID is not a valid 12-digit account ID."
  exit 1
fi

# Validation du format de l'URI avant push
if ! echo "${IMAGE_URI}" | grep -qE '^[0-9]{12}\.dkr\.ecr\.[a-z0-9-]+\.
    amazonaws\.com/.+$'; then
  echo "::error::Constructed IMAGE_URI is malformed: '${IMAGE_URI}'"
  exit 1
fi
```

Listing 4.8 – Validation des secrets (extrait)

Pourquoi ces validations ? Un secret vide ou malconforme pourrait produire une image avec un tag invalide, qui serait ensuite commite dans le manifest Kubernetes, causant un echec de deploiement en cascade.

4.3.3 Job 3 : Mise a jour GitOps

```bash
# 1. Substitution dans le fichier
sed -i "s|image: ${IMAGE_URI}:.*|image: ${FULL_IMAGE}|" k8s/deployment.
    yaml

# 2. Verification que la substitution a fonctionne
if ! grep -qF "image: ${FULL_IMAGE}" k8s/deployment.yaml; then
```

```
  echo "::error::Verification failed - image not found in deployment.yaml"
  exit 1
fi

# 3. Commit avec [skip ci] pour eviter la boucle infinie
git commit -m "chore(gitops): update image tag to ${IMAGE_TAG} [skip ci]"
```

Listing 4.9 – Mise a jour du manifest avec verification

Chapitre 5

Processus de Deploiement

5.1 Procedure de deploiement initial

5.1.1 Phase 1 : Provisionnement AWS (Terraform)

```
# 1. Configuration
cd terraform
cp terraform.tfvars.example terraform.tfvars
# Editer terraform.tfvars : key_name, allowed_cidr, aws_region

# 2. Initialisation et validation
terraform init
terraform fmt -check
terraform validate

# 3. Planification (verification avant execution)
terraform plan -out=tfplan

# 4. Application
terraform apply tfplan
```

Listing 5.1 – Commandes Terraform

Duree estimee : 3-5 minutes (creation EC2, VPC, etc.)
Outputs generes :

- `master_public_ip` : IP du noeud control plane
- `worker_public_ip` : IP du noeud worker
- `app_url` : URL d'acces a l'application
- `ansible/inventory.ini` : Inventaire genere automatiquement

5.1.2 Phase 2 : Configuration du cluster (Ansible)

```
# 1. Definition des secrets (ne jamais commiter ces valeurs)
export POSTGRES_PASSWORD="$(openssl rand -base64 32)"
export JWT_SECRET="$(openssl rand -base64 32)"
export AWS_ACCOUNT_ID="883999922093"

# 2. Execution du playbook
cd ansible
```

```
ansible-playbook -i inventory.ini install_k3s.yml

# Duree estimee : 10-15 minutes
```

Listing 5.2 – Execution Ansible

Etapes executees par Ansible :

1. Installation des paquets (curl, git, awscli, docker)
2. Configuration du swap (2 Go) pour eviter les OOM kills
3. Installation de k3s sur le master
4. Jointure du worker au cluster
5. Installation d'ArgoCD
6. Creation des secrets Kubernetes
7. Deploiement de l'application ArgoCD

5.1.3 Phase 3 : Verification

```
# Connexion au master
ssh -i ~/.ssh/key.pem ubuntu@<MASTER_IP>

# Verification du cluster
sudo k3s kubectl get nodes
# Attendu : 2 noeuds en status Ready

# Verification des pods
sudo k3s kubectl get pods -n default
# Attendu : 2 pods inforganizer Running, 1 pod postgres Running

# Verification d'ArgoCD
sudo k3s kubectl get application -n argocd
# Attendu : inforganizer Synced Healthy

# Test de l'application
curl http://<MASTER_IP>:30080/api/health
# Attendu : {"status":"ok"}
```

Listing 5.3 – Commandes de verification

5.2 Deploiement continu automatise

Apres la configuration initiale, tout deploiement ulterieur est automatique :

1. Le developpeur modifie le code et pousse sur `main`.
2. GitHub Actions execute les tests, build l'image, la pousse sur ECR.
3. Le manifest `k8s/deployment.yaml` est mis a jour avec le nouveau tag.
4. ArgoCD detecte le changement (polling toutes les 3 secondes par defaut).
5. ArgoCD applique le nouveau deployment au cluster.
6. Kubernetes effectue un RollingUpdate : nouveau pod cree, ancien termine.

Temps mesure : 1 minute 22 secondes du commit a l'application deployee (voir Figure 6.10).

Chapitre 6

Preuves et Validation

Cette section presente les preuves d'execution collectees lors du deploiement reel du systeme.

6.1 Infrastructure AWS

6.1.1 Instances EC2

Deux instances EC2 de type `c7i-flex.large` ont ete provisionnees :

— `inforganizer-master` : Noeud control plane k3s

— `inforganizer-worker` : Noeud worker pour les workloads

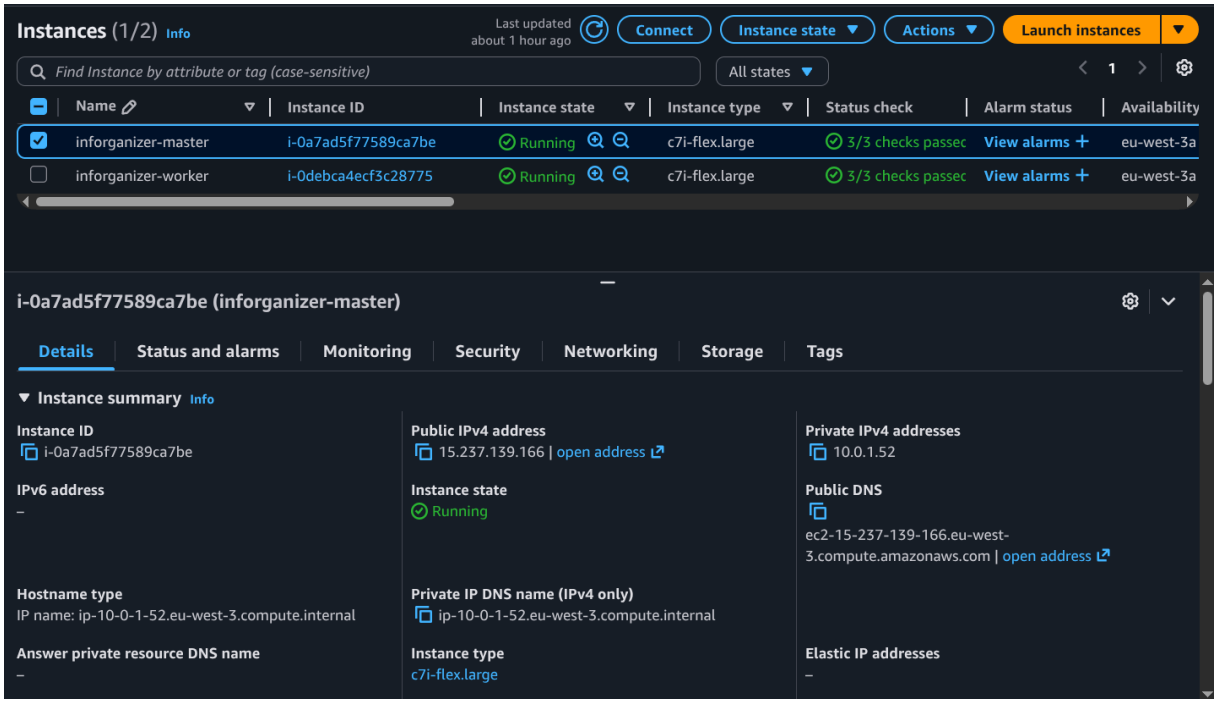

FIGURE 6.1 – Instances EC2 provisionnees sur AWS

6.1.2 Security Groups

Les groupes de securite ont ete configures pour restreindre l'acces :

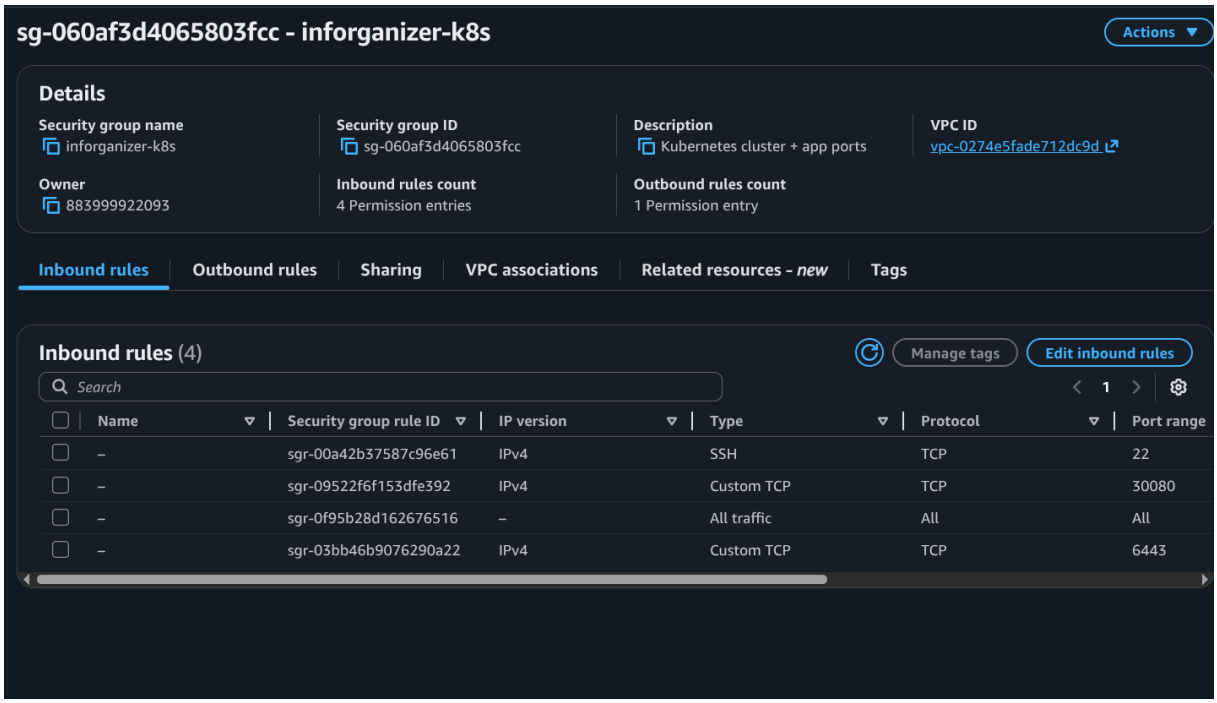

FIGURE 6.2 – Configuration des Security Groups AWS

6.1.3 Configuration IAM OIDC

Le role `AWS_GITHUB_ROLE_ARN` a ete configure avec :

— Trust relationship : GitHub OIDC provider
— Permissions : AmazonEC2ContainerRegistryPowerUser (lecture/ecriture ECR)
— Session duration : 1 heure maximum

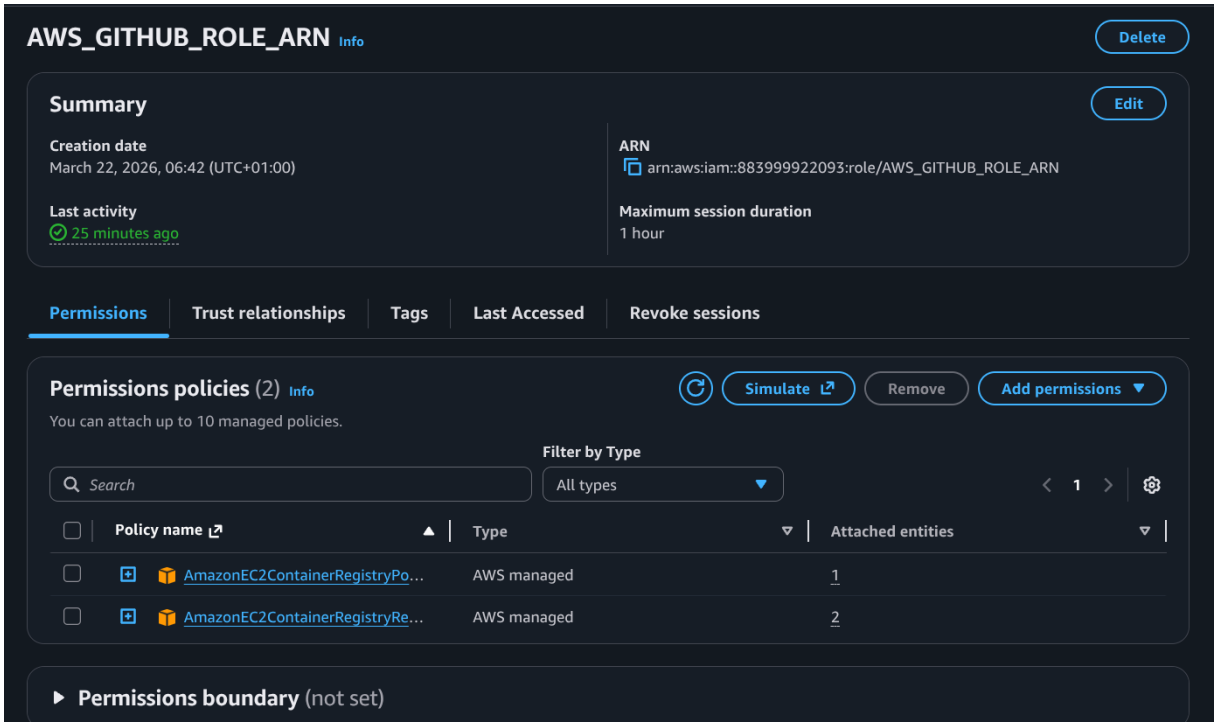

FIGURE 6.3 – Configuration du role IAM OIDC pour GitHub Actions

6.2 Provisionnement et Configuration

6.2.1 Terraform Apply

L'infrastructure a ete provisionnee avec succes via Terraform :

```
➜  inforganizer-devopss git:(main) ✗ cd terraform
➜  terraform git:(main) ✗ terraform apply -auto-approve
data.aws_ami.ubuntu: Reading...
aws_vpc.inforganizer: Refreshing state... [id=vpc-0274e5fade712dc9d]
data.aws_ami.ubuntu: Read complete after 1s [id=ami-0ad53ba8e49f41b76]
aws_internet_gateway.igw: Refreshing state... [id=igw-0c5a31ecd816b06b6]
aws_subnet.public: Refreshing state... [id=subnet-03d810bacf886a4b3]
aws_security_group.k8s: Refreshing state... [id=sg-060af3d4065803fcc]
aws_route_table.public: Refreshing state... [id=rtb-05d18fd9abf1e5dd5]
aws_instance.master: Refreshing state... [id=i-0a7ad5f77589ca7be]
aws_instance.worker: Refreshing state... [id=i-0debca4ecf3c28775]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-0865919cc01cab0e0]
null_resource.ansible_inventory: Refreshing state... [id=5508724771644451635]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

app_url = "http://15.237.139.166:30080"
master_public_ip = "15.237.139.166"
worker_public_ip = "35.180.202.222"
➜  terraform git:(main) ✗
```

FIGURE 6.4 – Execution reussie de Terraform apply

6.2.2 Ansible Playbook

Le playbook Ansible a configure le cluster k3s avec succes :

```
TASK [Apply ArgoCD install manifest (server-side fallback)] ***********************************************************
changed: [15.237.139.166]

TASK [Wait for ArgoCD server to be ready] ******************************************************************************
changed: [15.237.139.166]

TASK [Copy ArgoCD application manifest to master] **********************************************************************
ok: [15.237.139.166]

TASK [Apply ArgoCD application] ****************************************************************************************
changed: [15.237.139.166]

TASK [Trigger initial ArgoCD sync by annotating app] *******************************************************************
changed: [15.237.139.166]

TASK [Print app URL] ***************************************************************************************************
ok: [15.237.139.166] => {
    "msg": "GitOps is active. ArgoCD will reconcile from GitHub and Inforganizer is exposed at http://15.237.139.166:30080\n"
}

PLAY RECAP *************************************************************************************************************
15.237.139.166             : ok=35   changed=11   unreachable=0    failed=0    skipped=3    rescued=1    ignored=0
35.180.202.222             : ok=13   changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
```

FIGURE 6.5 – Recap de l'execution du playbook Ansible

6.3 Cluster Kubernetes

6.3.1 Etat des noeuds

```
ubuntu@ip-10-0-1-52:~/.ssh$ sudo k3s kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE    VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION
CONTAINER-RUNTIME
ip-10-0-1-52   Ready    control-plane   3h4m   v1.34.5+k3s1   10.0.1.52     <none>        Ubuntu 22.04.5 LTS   6.8.0-1050-aws
containerd://2.1.5-k3s1
ubuntu@ip-10-0-1-52:~/.ssh$ sudo k3s kubectl get application -n argocd
NAME           SYNC STATUS   HEALTH STATUS
inforganizer   Synced        Healthy
ubuntu@ip-10-0-1-52:~/.ssh$ sudo k3s kubectl get pods -n default
NAME                                                READY   STATUS    RESTARTS       AGE
argocd-application-controller-0                     1/1     Running   0              107m
argocd-applicationset-controller-7dc85c9598-lsmzb   1/1     Running   1 (107m ago)   127m
argocd-dex-server-7d88b5b798-xxsfw                  1/1     Running   2 (122m ago)   127m
argocd-notifications-controller-5f95df7f8c-rtw9p    1/1     Running   1 (121m ago)   127m
argocd-redis-65b5bbd8cb-f47mj                       1/1     Running   0              127m
argocd-repo-server-b6dc69ff-l62qr                   1/1     Running   0              51m
argocd-server-58dbdffdcd-9ctmd                      1/1     Running   0              126m
inforganizer-5f57b75f99-qqxjx                       1/1     Running   0              31m
inforganizer-5f57b75f99-vk9wr                       1/1     Running   0              31m
postgres-545cf88b5d-lff8l                           1/1     Running   0              69m
ubuntu@ip-10-0-1-52:~/.ssh$
```

FIGURE 6.6 – Noeuds Kubernetes en etat Ready

6.3.2 Pods applicatifs

Deux replicas de l'application sont en execution :

- `inforganizer-5f57b75f99-qqxjx` : Running, Ready 1/1
- `inforganizer-5f57b75f99-vk9wr` : Running, Ready 1/1

```
➜  inforganizer-devopss git:(main) ✗ ssh -i "$SSH_KEY" ubuntu@"$MASTER_IP" 'sudo k3s kubectl get pods -n default -l app=inforganizer -o wide'
NAME                           READY   STATUS    RESTARTS   AGE   IP           NODE           NOMINATED NODE   READINESS GATES
inforganizer-5f57b75f99-qqxjx   1/1     Running   0          32m   10.42.0.51   ip-10-0-1-52   <none>           <none>
inforganizer-5f57b75f99-vk9wr   1/1     Running   0          32m   10.42.0.50   ip-10-0-1-52   <none>           <none>
➜  inforganizer-devopss git:(main) ✗
```

FIGURE 6.7 – Pods applicatifs en execution

6.3.3 Services exposes

Le service `inforganizer` est accessible sur le port 30080 depuis l'exterieur. Le service `postgres` reste interne au cluster (ClusterIP).

```
➜  inforganizer-devopss git:(main) ✗ ssh -i "$SSH_KEY" ubuntu@"$MASTER_IP" 'sudo k3s kubectl get svc -n default'
NAME                                      TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
argocd-applicationset-controller          ClusterIP   10.43.21.20     <none>        7000/TCP,8080/TCP            132m
argocd-dex-server                         ClusterIP   10.43.104.247   <none>        5556/TCP,5557/TCP,5558/TCP   132m
argocd-metrics                            ClusterIP   10.43.185.169   <none>        8082/TCP                     132m
argocd-notifications-controller-metrics   ClusterIP   10.43.139.194   <none>        9001/TCP                     132m
argocd-redis                              ClusterIP   10.43.20.70     <none>        6379/TCP                     132m
argocd-repo-server                        ClusterIP   10.43.241.141   <none>        8081/TCP,8084/TCP            131m
argocd-server                             ClusterIP   10.43.41.161    <none>        80/TCP,443/TCP               131m
argocd-server-metrics                     ClusterIP   10.43.229.185   <none>        8083/TCP                     131m
inforganizer                              NodePort    10.43.205.188   <none>        80:30080/TCP                 71m
kubernetes                                ClusterIP   10.43.0.1       <none>        443/TCP                      3h7m
postgres                                  ClusterIP   10.43.12.145    <none>        5432/TCP                     71m
➜  inforganizer-devopss git:(main) ✗
```

FIGURE 6.8 – Services Kubernetes exposes

6.3.4 Test de sante de l'API

L'endpoint de sante de l'application repond correctement :

```
→ inforganizer-devopss git:(main) ✗ curl -s "http://$MASTER_IP:30080/api/health" && echo
{"status":"ok"}
→ inforganizer-devopss git:(main) ✗
```

FIGURE 6.9 – Test de sante de l'API - reponse OK

6.4 Pipeline CI/CD

6.4.1 Execution reussie

Le workflow `CI/CD (GitOps + ECR)` s'est execute avec succes :

— **Commit** : feat(ci) : enhance CI/CD pipeline (f650335)

— **Duree totale** : 1 minute 22 secondes

— **Jobs** : Install and Test (35s) + Build and Push (35s) + Update GitOps (5s)

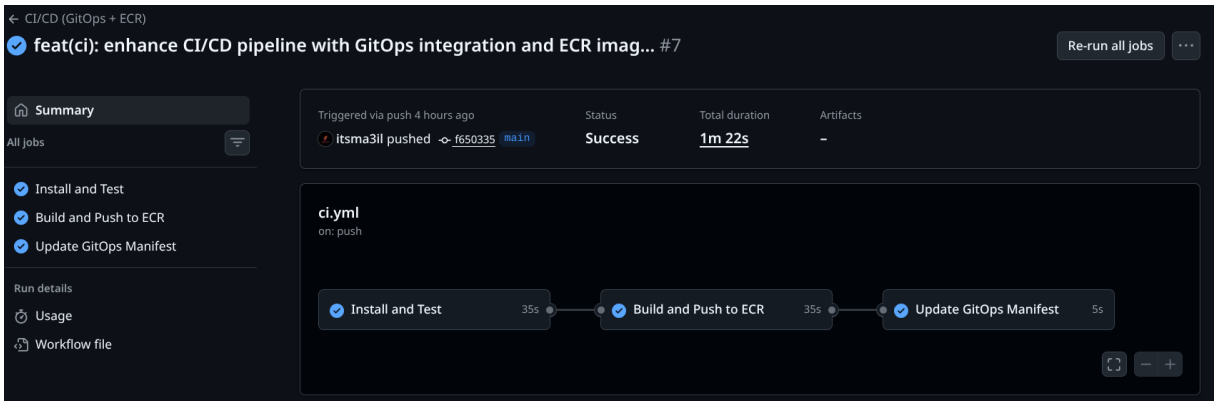

FIGURE 6.10 – Pipeline CI/CD execute avec succes

6.4.2 Image publiee sur ECR

L'image a ete publiee avec deux tags :

— `sha-94bbf8f77cb9` : Tag immutable lie au commit

— `latest` : Tag mobile pour faciliter les tests

Taille de l'image : 49.31 Mo

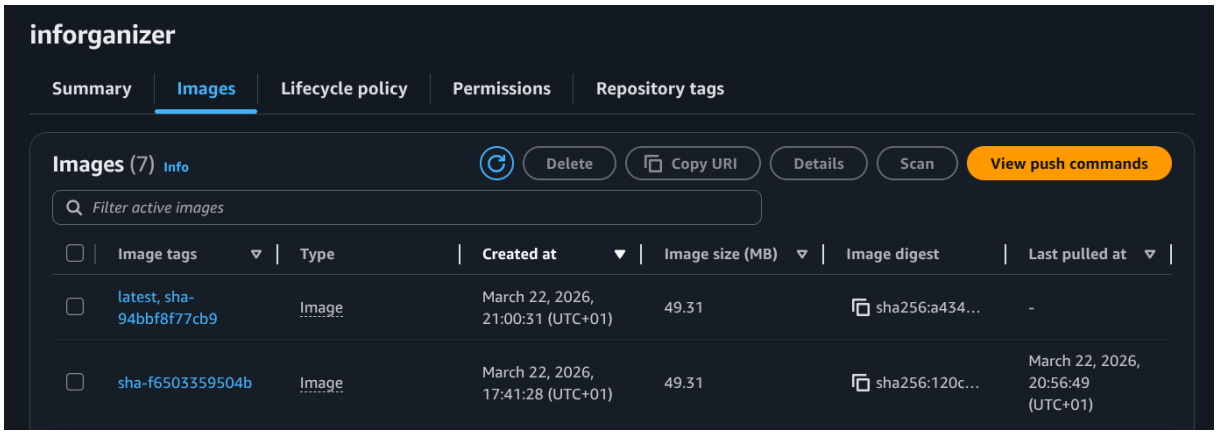

FIGURE 6.11 – Image Docker publiee sur AWS ECR

6.5 GitOps ArgoCD

6.5.1 Synchronisation

L'application `inforganizer` dans ArgoCD affiche :

— **SYNC STATUS** : Synced (le cluster correspond a Git)

— **HEALTH STATUS** : Healthy (tous les pods sont operationnels)

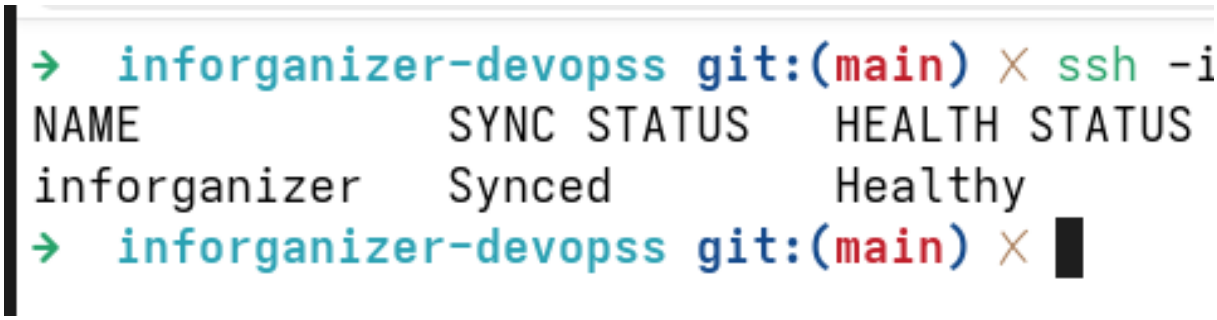

FIGURE 6.12 – ArgoCD - Application synchronisee et saine

6.6 Tableau recapitulatif des validations

ID	Element valide	Statut	Figure
PREUVE_01	Instances EC2 creees	OK	Fig. 6.1
PREUVE_02	Security Groups configures	OK	Fig. 6.2
PREUVE_03	IAM OIDC fonctionnel	OK	Fig. 6.3
PREUVE_04	Terraform apply reussi	OK	Fig. 6.4
PREUVE_05	Ansible playbook complet	OK	Fig. 6.5
PREUVE_06	Noeuds k3s Ready	OK	Fig. 6.6
PREUVE_07	Pods Running (2 replicas)	OK	Fig. 6.7
PREUVE_08	Service NodePort expose	OK	Fig. 6.8
PREUVE_09	API Health OK	OK	Fig. 6.9
PREUVE_10	Pipeline CI/CD vert	OK	Fig. 6.10
PREUVE_11	Image publiee sur ECR	OK	Fig. 6.11
PREUVE_12	ArgoCD Synced/Healthy	OK	Fig. 6.12

Table 6.1 – Synthese des validations

Chapitre 7

Discussion

7.1 Points forts de l'implementation

7.1.1 Automatisation complete

Le systeme atteint l'objectif de zero intervention manuelle apres la configuration initiale :

— Temps de deploiement : 1 min 22s (du commit a la production)

— Zero credentials statiques grace a OIDC

— Self-healing via ArgoCD

7.1.2 Securite en profondeur

Plusieurs couches de securite ont ete implementees :

1. **Infrastructure** : Security Groups restrictifs, SSH limite a une IP.
2. **CI/CD** : OIDC elimine le risque de fuite de credentials.
3. **Container** : Utilisateur non-root, image minimale Alpine.
4. **Kubernetes** : Secrets pour les donnees sensibles, probes de sante.

7.1.3 Haute disponibilite

— 2 replicas avec anti-affinite : un noeud peut tomber sans impact utilisateur.

— RollingUpdate avec maxUnavailable=0 : zero downtime lors des deploiements.

— Base PostgreSQL sur volume persistant : les donnees survivent aux redemarrages.

7.2 Limites identifiees

7.2.1 Single Point of Failure : Control Plane

Le cluster utilise un seul noeud master. En cas de panne de ce noeud :

— L'API Kubernetes devient inaccessible.

— Les pods existants continuent de fonctionner (sur le worker).

— Aucun nouveau deploiement ni scaling n'est possible.

Solution recommandee : Deployer k3s en mode HA avec 3 noeuds control plane et un load balancer.

7.2.2 Absence de monitoring

Le systeme ne dispose pas de :

— Collecte de metriques (CPU, memoire, requetes HTTP).

— Alerting en cas d'anomalie.

— Dashboard de visualisation.

Solution recommandee : Deployer la stack Prometheus + Grafana + AlertManager.

7.2.3 Gestion des secrets

Les secrets sont injectes via variables d'environnement Ansible, ce qui presente des limites :

— Pas de rotation automatique.

— Pas d'audit trail des acces aux secrets.

— Secrets en clair dans la memoire des pods.

Solution recommandee : Integrer HashiCorp Vault avec External Secrets Operator.

7.2.4 Backup de la base de donnees

Aucun mecanisme de sauvegarde automatique n'est en place. En cas de corruption ou suppression accidentelle :

— Perte totale des donnees utilisateurs.

— Pas de Point-in-Time Recovery possible.

Solution recommandee : Implementer pg_dump periodique vers S3, ou utiliser un operateur PostgreSQL (CloudNativePG, Zalando).

7.3 Ameliorations proposees

Priorite	Amelioration	Effort	Impact
1	Monitoring Prometheus/Grafana	Moyen	Haut
2	Backup PostgreSQL automatise	Faible	Haut
3	Cluster k3s HA (3 masters)	Eleve	Moyen
4	Vault pour secrets	Moyen	Moyen
5	TLS avec cert-manager	Moyen	Moyen
6	Deploiement Canary	Eleve	Faible

Table 7.1 – Ameliorations proposees par priorite

Chapitre 8

Conclusion

8.1 Synthese des realisations

Ce projet a permis de concevoir et deployer une infrastructure DevOps complete, demontrant la mise en oeuvre concrete des pratiques modernes du genie logiciel :

— **Infrastructure as Code** : L'ensemble de l'infrastructure AWS est defini dans des fichiers Terraform versiones, permettant une recreation identique en moins de 30 minutes.

— **Configuration as Code** : Les playbooks Ansible documentent et automatisent chaque etape de configuration, eliminant les procedures manuelles sujettes aux erreurs.

— **GitOps** : Git constitue la source de verite unique pour l'etat du systeme. Chaque modification est tracee, versionee, et reversible.

— **Integration Continue** : Chaque commit declenche automatiquement tests, build et deploiement, garantissant un feedback rapide aux developpeurs.

— **Deploiement Continu** : L'application est deployee en production en 1 minute 22 secondes apres un commit, sans intervention humaine.

8.2 Competences acquises

Ce projet a permis de developper des competences directement applicables en contexte professionnel :

1. Conception d'architectures cloud-native
2. Provisionnement d'infrastructure avec Terraform
3. Configuration automatisee avec Ansible
4. Orchestration de conteneurs avec Kubernetes
5. Mise en place de pipelines CI/CD
6. Implementation du paradigme GitOps
7. Securisation des workflows (OIDC, secrets)

8.3 Perspectives

Les extensions naturelles de ce travail incluent :

— **Observabilite** : Integration de Prometheus, Grafana et Loki pour le monitoring et le logging centralise.

— **Securite renforcee** : Ajout de Network Policies Kubernetes, scan d'images avec Trivy, gestion des secrets avec Vault.

— **Multi-environnements** : Separation staging/production avec promotion automatisee.

— **Scalabilite** : Auto-scaling horizontal base sur les metriques applicatives.

8.4 Mot de fin

Ce projet demontre qu'il est possible, avec les outils modernes du DevOps, de mettre en place une infrastructure de niveau professionnel dans un contexte academique. L'automatisation, la tracabilite et la reproductibilite ne sont plus des luxes reserves aux grandes entreprises, mais des pratiques accessibles a tout projet logiciel.

Bibliographie

1. Kubernetes Documentation. https://kubernetes.io/docs/
2. k3s Documentation - Lightweight Kubernetes. https://docs.k3s.io/
3. Terraform AWS Provider Documentation. https://registry.terraform.io/providers/hashicorp/aws/latest/docs
4. Ansible Documentation. https://docs.ansible.com/
5. GitHub Actions Documentation. https://docs.github.com/en/actions
6. ArgoCD - Declarative GitOps CD for Kubernetes. https://argo-cd.readthedocs.io/
7. Docker Documentation. https://docs.docker.com/
8. AWS ECR Documentation. https://docs.aws.amazon.com/ecr/
9. Kim, G., Humble, J., Debois, P., Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press.
10. Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (2016). *Site Reliability Engineering*. O'Reilly Media.
11. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. ACM Queue.
12. Weaveworks (2017). *GitOps - Operations by Pull Request*. https://www.weave.works/technologies/gitops/

Annexe A

Annexes

A.1 Structure complete du projet

```
inforganizer-devopss/
|-- .github/
|   `-- workflows/
|       `-- ci.yml                  # Pipeline CI/CD
|-- ansible/
|   |-- install_k3s.yml             # Playbook principal
|   |-- inventory.ini               # Genere par Terraform
|   `-- templates/
|       `-- postgres-secret.yaml.j2
|-- argocd/
|   `-- inforganizer-app.yaml       # Definition Application ArgoCD
|-- client/                         # Frontend HTML/CSS/JS
|-- k8s/
|   |-- deployment.yaml             # Deployment + init container
|   |-- service.yaml                # Service NodePort
|   |-- postgres.yaml               # Deployment + Service PostgreSQL
|   `-- pvc.yaml                    # PersistentVolumeClaim
|-- server/
|   |-- server.js                   # API Express
|   |-- database.js                 # Schema et pool PostgreSQL
|   `-- auth.js                     # JWT et middleware
|-- terraform/
|   |-- main.tf                     # Infrastructure AWS
|   |-- terraform.tfvars.example    # Template de configuration
|-- tests/                          # Tests Jest
|-- Dockerfile                      # Build multi-stage
|-- docker-compose.yml              # Developpement local
|-- package.json
`-- pnpm-lock.yaml
```

Listing A.1 – Arborescence du projet

A.2 Commandes de reference

A.2.1 Terraform

```
terraform init          # Telecharge les providers
```

```
terraform fmt           # Formate le code
terraform validate      # Verifie la syntaxe
terraform plan          # Affiche les changements prevus
terraform apply         # Execute les changements
terraform destroy       # Detruit l'infrastructure
terraform output        # Affiche les outputs
```

A.2.2 Ansible

```
ansible-playbook -i inventory.ini playbook.yml        # Execution
ansible-playbook -i inventory.ini playbook.yml --check # Dry-run
ansible-playbook -i inventory.ini playbook.yml -vvv    # Debug
ansible all -i inventory.ini -m ping                   # Test
    connectivite
```

A.2.3 Kubernetes

```
kubectl get nodes -o wide                    # Liste des noeuds
kubectl get pods -A                          # Tous les pods
kubectl get pods -n default -l app=inforganizer
kubectl describe pod <name>                  # Details d'un pod
kubectl logs <pod> -f                        # Logs en temps reel
kubectl exec -it <pod> -- /bin/sh            # Shell interactif
kubectl rollout status deployment/inforganizer
kubectl rollout history deployment/inforganizer
kubectl rollout undo deployment/inforganizer # Rollback
```

A.2.4 ArgoCD

```
kubectl get application -n argocd
kubectl describe application inforganizer -n argocd
# Ou via CLI argocd :
argocd app list
argocd app sync inforganizer
argocd app history inforganizer
```

A.2.5 Docker

```
docker build -t inforganizer:local .
docker run -p 3000:3000 --env-file .env inforganizer:local
docker images | grep inforganizer
docker system prune -a                       # Nettoyage
```

A.3 Variables d'environnement requises

Variable	**Contexte**	**Description**
POSTGRES_PASSWORD	Ansible	Mot de passe PostgreSQL
JWT_SECRET	Ansible	Cle de signature JWT
AWS_ACCOUNT_ID	GitHub Secrets	ID du compte AWS (12 chiffres)
AWS_GITHUB_ROLE_ARN	GitHub Secrets	ARN du role IAM OIDC

Table A.1 – Variables d'environnement et secrets